Matplotlib

Irene Siragusa, PhD

CHILAB
Università degli Studi di Palermo

Matplotlib

- **Matplotlib** is a comprehensive library for creating static, animated, and interactive visualizations in Python.
- **matplotlib.pyplot** is a state-based interface to matplotlib. It provides an implicit way of plotting. It also opens figures on your screen, and acts as the figure GUI manager. It can be useful for plot graphs and add legend and graphics.
- It is compatible with other libraries as pandas

- Other useful library is **seaborn**

Matplotlib

- Pandas is the reference framework for data manipulation in Python

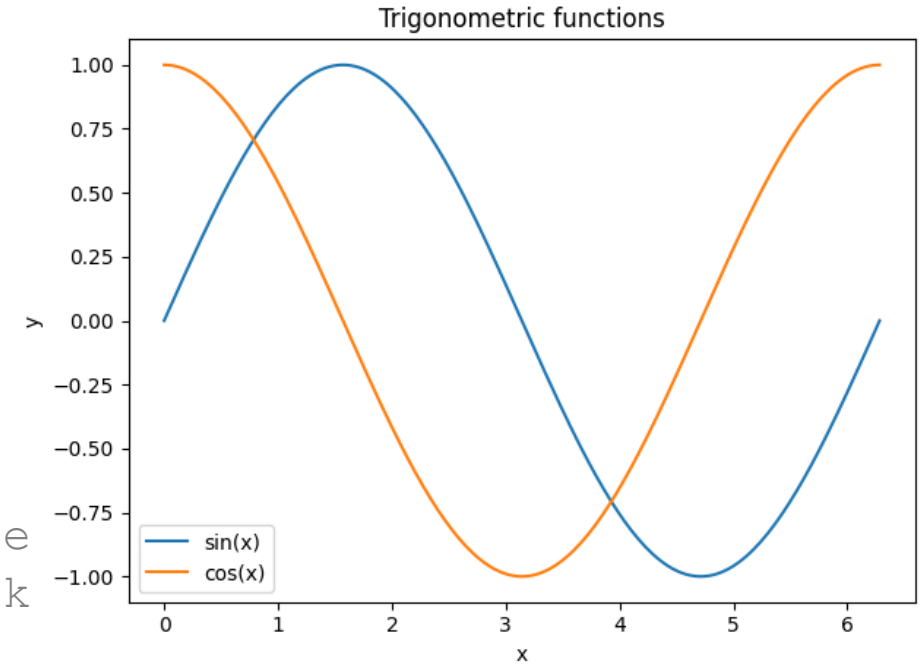
```
# in terminal with the enviromment activated
pip install matplotlib

# in notebook cell
! pip install matplotlib

import matplotlib.pyplot as plt
```

Matplotlib

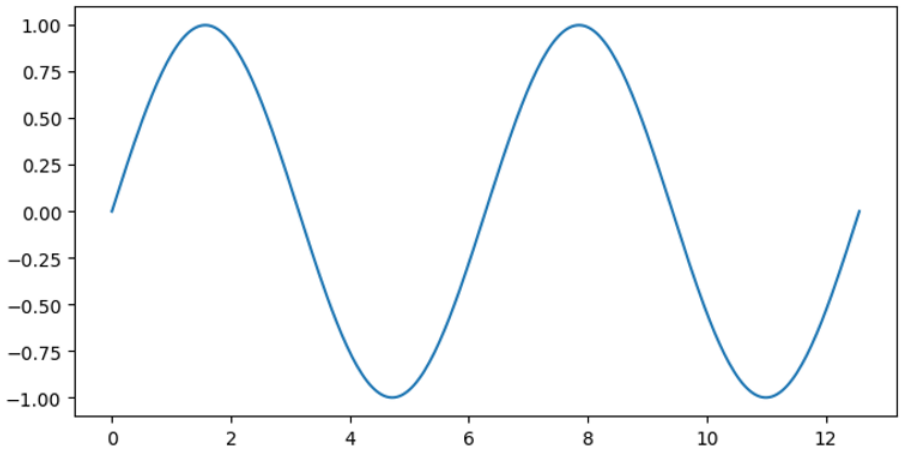
```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 2*np.pi, 200)
plt.plot(x, np.sin(x), label='sin(x)')
plt.plot(x, np.cos(x), label='cos(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Trigonometric functions')
plt.legend()
plt.savefig('trig.png', dpi=150) # save on file
plt.show() # open window for dispay in notebook
plt.close()# empty memory
```



Matplotlib – plot

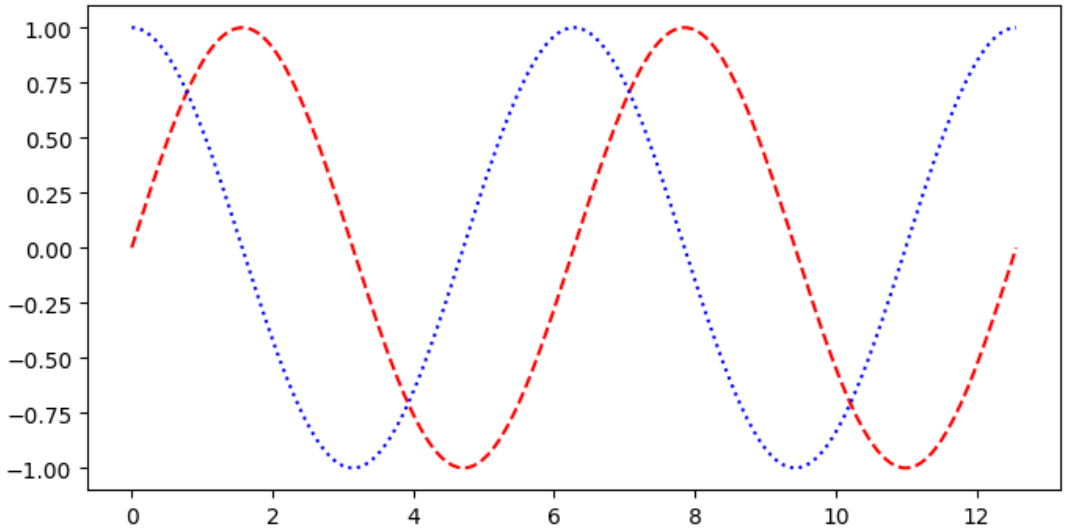
```
fig, ax = plt.subplots(figsize=(8, 4))
x = np.linspace(0, 4*np.pi, 300)

# base form
ax.plot(x, np.sin(x))
```



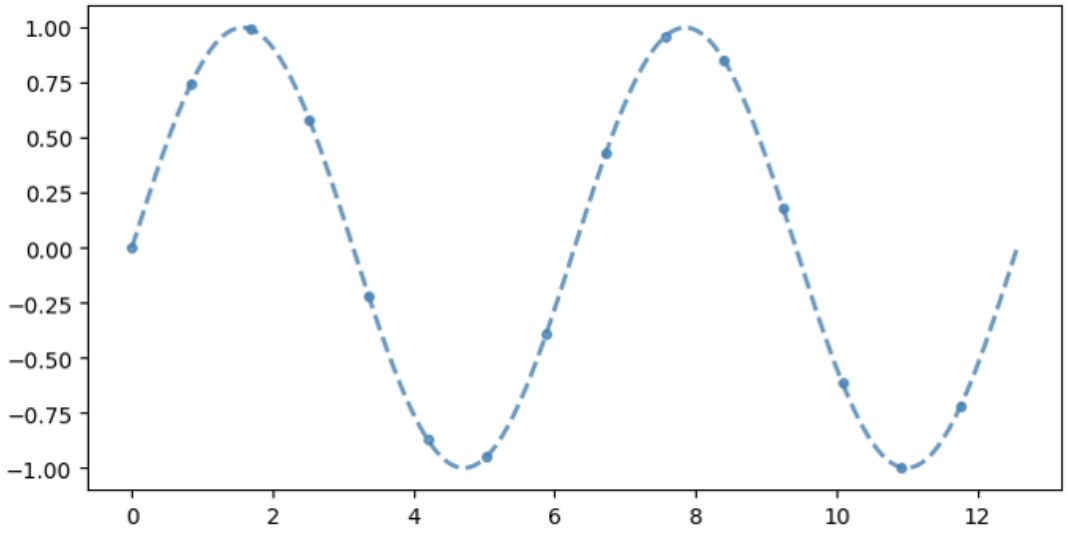
Matplotlib – plot

```
fig, ax = plt.subplots(figsize=(8, 4))
x = np.linspace(0, 4*np.pi, 300)
# color-style-marker
ax.plot(x, np.sin(x), 'r--') # red, dashed
ax.plot(x, np.cos(x), 'b:')  # blue, points
```



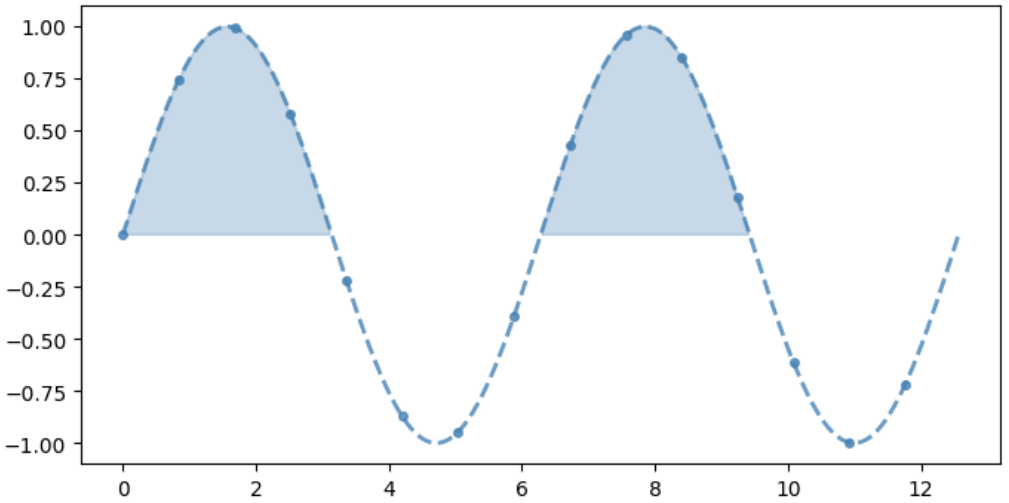
Matplotlib – plot

```
fig, ax = plt.subplots(figsize=(8, 4))
x = np.linspace(0, 4*np.pi, 300)
# explicit form
ax.plot(x, np.sin(x),
            color='steelblue',
            linestyle='--',
            linewidth=2,
            marker='o',
            markersize=4,
            markevery=20,
            alpha=0.8,
            label='sin(x)')
```



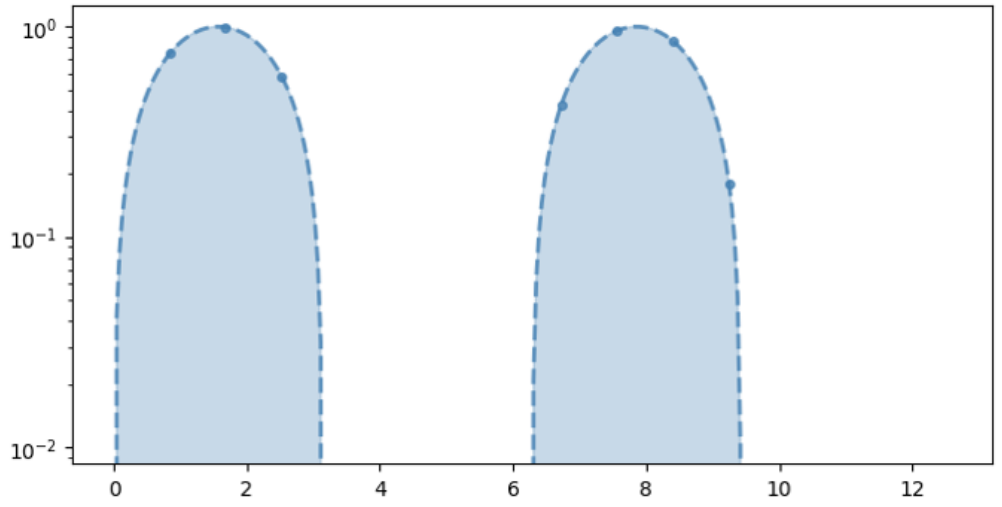
Matplotlib – plot

```
fig, ax = plt.subplots(figsize=(8, 4))
x = np.linspace(0, 4*np.pi, 300)
# explicit form
ax.plot(x, np.sin(x),
            color='steelblue',
            linestyle='--', linewidth=2,
            marker='o', markersize=4,
            markevery=20, alpha=0.8,
            label='sin(x)')
# fill area
ax.fill_between(x, np.sin(x), 0,
                   where=(np.sin(x)>0),
                   alpha=0.3, color='steelblue')
```



Matplotlib – plot

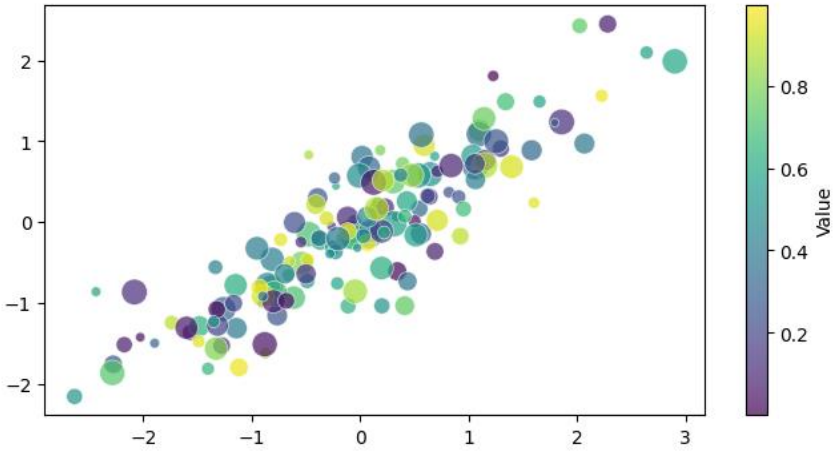
```
fig, ax = plt.subplots(figsize=(8, 4))
x = np.linspace(0, 4*np.pi, 300)
# explicit form
ax.plot(x, np.sin(x),
            color='steelblue',
            linestyle='--', linewidth=2,
            marker='o', markersize=4,
            markevery=20, alpha=0.8,
            label='sin(x)')
# fill area
ax.fill_between(x, np.sin(x), 0,
                   where=(np.sin(x)>0),
                   alpha=0.3, color='steelblue')
# logarithm scale
ax.set_yscale('log') # 'symlog', 'logit'
```



Matplotlib – scatter

```
fig, ax = plt.subplots(figsize=(8, 4))

rng = np.random.default_rng(17)
N = 150
x_s = rng.standard_normal(N)
y_s = x_s*0.8 + rng.standard_normal(N)*0.5
col = rng.uniform(0, 1, N) # colormap values
siz = rng.uniform(20, 200, N) # marker size
sc = ax.scatter(x_s, y_s,
                c=col, # array color
                s=siz, # dimensions
                cmap='viridis', alpha=0.7,
                edgecolors='white', linewidths=0.5)
cbar = fig.colorbar(sc, ax=ax)
cbar.set_label('Value', fontsize=10)
```

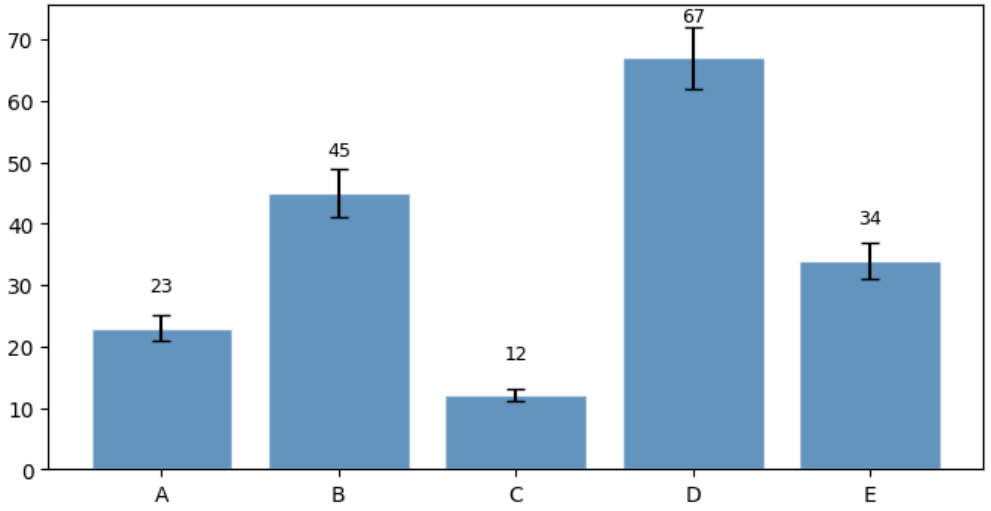


Matplotlib – plot vs scatter

- plot() is optimized for continuous lines and obtains good performances over a large data
- scatter() supports colors and dimensions per point
  (4D: x, y, color, size)

Matplotlib – bar

```
fig, ax = plt.subplots(figsize=(8, 4))
categories = ['A', 'B', 'C', 'D', 'E']
values = [23, 45, 12, 67, 34]
errors = [2, 4, 1, 5, 3]
# vertical error bars
bars = ax.bar(categories, values,
              color='steelblue',
              edgecolor='white',
              yerr=errors,
              capsize=4, alpha=0.85)
# labels over bars
for bar, v in zip(bars, values):
      ax.text(bar.get_x() + bar.get_width()/2,
              bar.get_height() + 6,
              str(v), ha='center', fontsize=9)
```

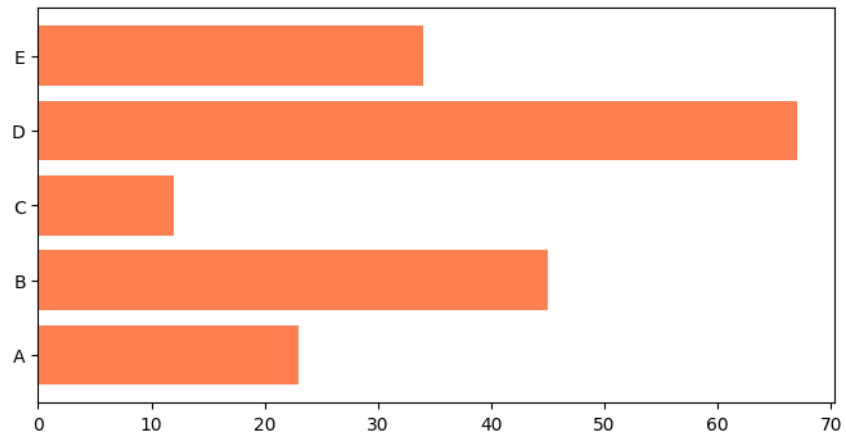


Matplotlib – barh

```
fig, ax = plt.subplots(figsize=(8, 4))


categories = ['A', 'B', 'C', 'D', 'E']
values = [23, 45, 12, 67, 34]


# horizontal bars
ax.barh(categories, values, color='coral')
```



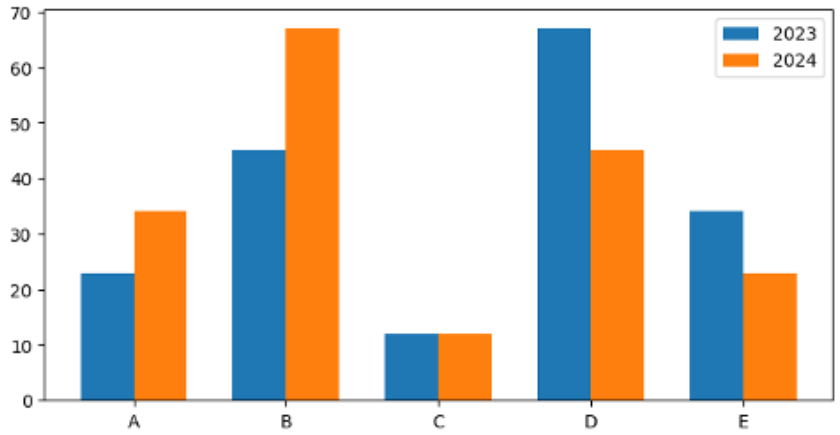
Matplotlib – bar

```
fig, ax = plt.subplots(figsize=(8, 4))

categories = ['A', 'B', 'C', 'D', 'E']
values = [23, 45, 12, 67, 34]



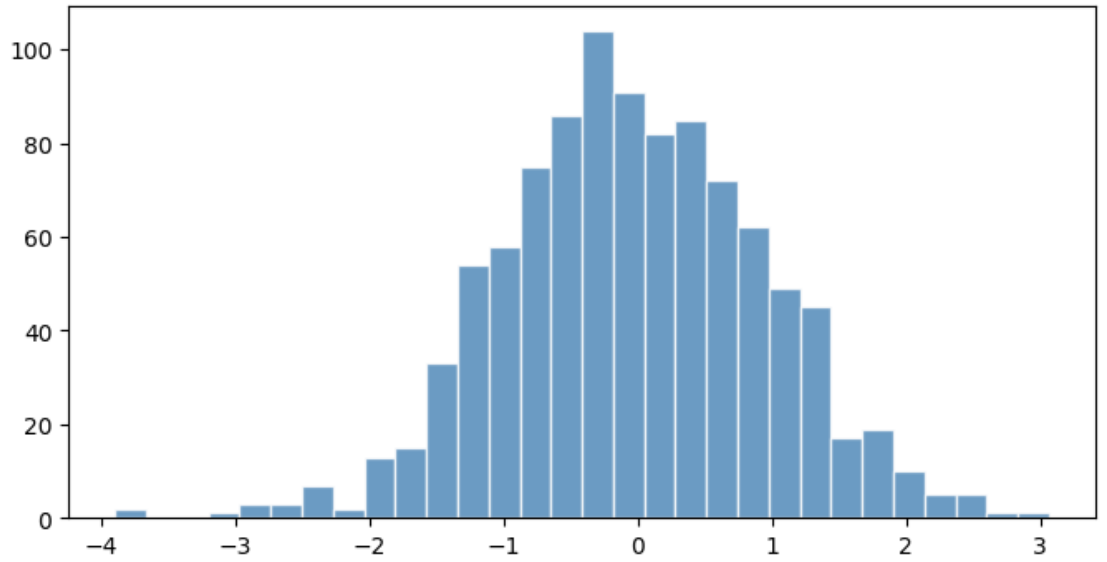
# Grouped bar chart
x_pos = np.arange(len(categories))
width = 0.35
ax.bar(x_pos - width/2, values, width, label='2023')
ax.bar(x_pos + width/2, values[::-1], width, label='2024')
ax.set_xticks(x_pos)
ax.set_xticklabels(categories)
ax.legend()
```



Matplotlib – hist

```
fig, ax = plt.subplots(figsize=(8, 4))

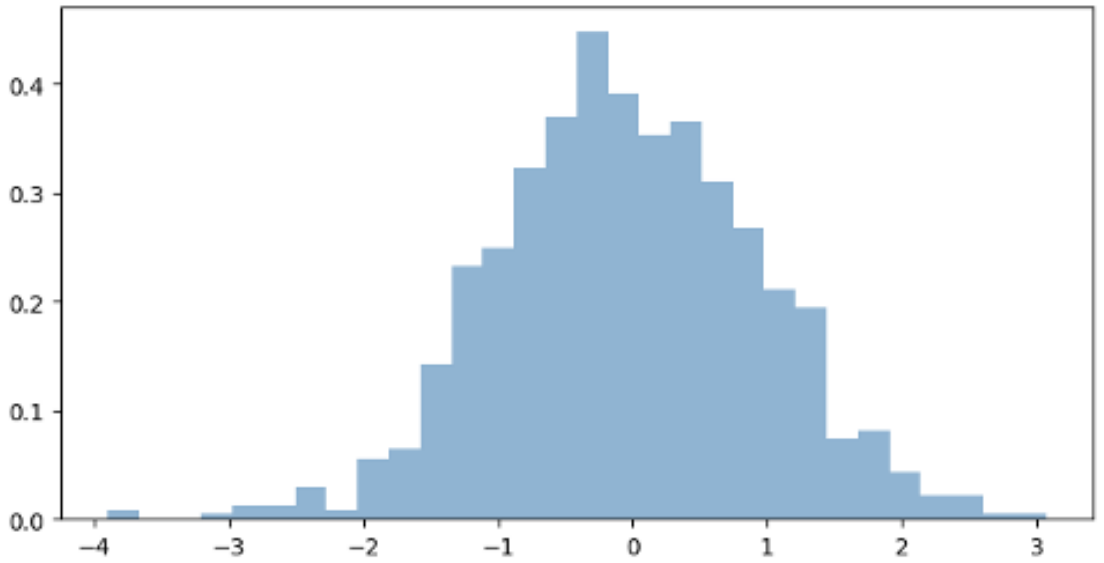
rng = np.random.default_rng(0)
data = rng.standard_normal(1000)
# base hist
ax.hist(data,
        bins=30,
        color='steelblue',
        edgecolor='white',
        alpha=0.8)
```



Matplotlib – hist

```
fig, ax = plt.subplots(figsize=(8, 4))

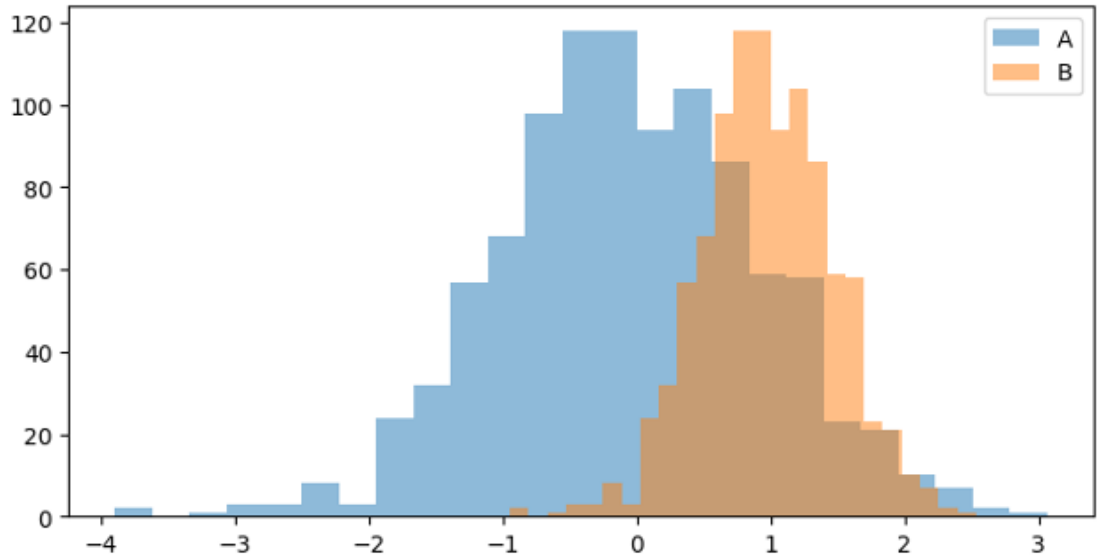
rng = np.random.default_rng(0)
data = rng.standard_normal(1000)
# Normalize to probability densities
ax.hist(data,
        bins=30,
        density=True,
        color='steelblue',
        alpha=0.6,
        label='data')
```



Matplotlib – hist

```
fig, ax = plt.subplots(figsize=(8, 4))

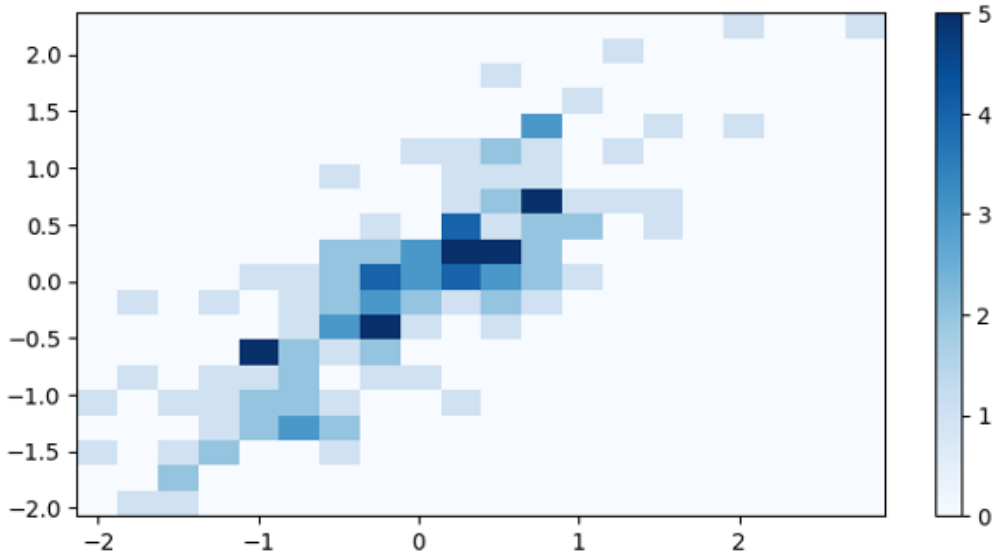
rng = np.random.default_rng(0)
data = rng.standard_normal(1000)
# comparison w/ two distributions
ax.hist(data, bins=25, alpha=0.5,
        label='A')
ax.hist(data*0.5+1, bins=25,
          alpha=0.5, label='B')
ax.legend()
```



Matplotlib – hist

```
# 2D histogram: density heatmap

fig, ax = plt.subplots(figsize=(8, 4))
rng = np.random.default_rng(42)
N = 150
x_s = rng.standard_normal(N)
y_s = x_s * 0.8 +
rng.standard_normal(N) * 0.5
h = ax.hist2d(x_s, y_s,
              bins=20, cmap='Blues')
fig.colorbar(h[3], ax=ax)
```



Matplotlib – colours

Syntax	Example	Description
HTML name	'steelblue', 'coral'	CSS/HTML names
HEX string	'#FF5733', '#2E86AB'	hexadecimal RGB
RGB tuple	(0.1, 0.6, 0.9)	0-1 values
RGBA tuple	(0.1, 0.6, 0.9, 0.5)	w/ alpha channel
Litteral	'r','g','b','c','m','y'	shortcut base
Greyscale	'0.5'	float 0-1

Matplotlib – linestyle and markers

linestyles	
linestyle	effect
'-' solid	continuous
'--' dashed	dashed
':' dotted	dotted
'-.' dashdot	dashhot
(0,(5,5))	custom dash

markers	
marker	symbol
'o'	circle
's'	square
'^'	triangle
'*'	star
'+'	cross / 'x'

Matplotlib – stylesheet

```
# list available styles
print(plt.style.available)
# Apply a global style
plt.style.use('seaborn-v0_8-whitegrid')
plt.style.use('ggplot')
# Global parameters rcParams
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12
```

Category	Examples	Usage
Sequential	viridis, plasma, inferno, cividis	ordered data 0→max
Divergent	RdBu, coolwarm, bwr, seismic	data 0-centered
Qualitative	tab10, Set1, Paired, Pastel1	discrete categories w/o intrinsic ordering
Ciclic	hsv, twilight	angles, phase

Matplotlib – object hierarchy

Object	Description
Figure	container: dimensions, background, figure title
Axes	coordinate system with axes, ticks, spine
Axis	single axis (x or y): scale, limits, ticks
Artist	everything that can be drawn: Line2D, Text, Patch…

Matplotlib – figure

```
fig, ax = plt.subplots(figsize=(8, 5), dpi=100)

fig.set_size_inches(12, 4) # resize

fig.suptitle('figure title', fontsize=16)

fig.tight_layout() # avoid overlaps

fig.savefig('out.png', dpi=150, bbox_inches='tight')

fig.savefig('out.pdf') # vector format
```

Matplotlib – axes

```
ax.set_title('Title', fontsize=14, pad=10)
ax.set_xlabel('X', fontsize=12); ax.set_ylabel('Y')
ax.set_xlim(0, 10)
ax.set_ylim(-1.5, 1.5)
ax.set_aspect('equal') # 1:1 ratio
ax.set_xticks([0, np.pi, 2*np.pi])
ax.set_xticklabels(['0', 'π', '2π'])
ax.tick_params(axis='both', direction='in', length=6)
ax.grid(True, which='major', ls='--', alpha=0.4)
ax.spines['top'].set_visible(False) # remove borders
```

Matplotlib – thicks, grid and spine

```
ax.minorticks_on()
ax.grid(True, which='minor', ls=':', alpha=0.2)
ax.tick_params(axis='x', labelrotation=45)


# Remove spines (graph borders)
ax.spines[['top','right']].set_visible(False)
ax.spines['left'].set_linewidth(1.5)
```

Matplotlib – twin axes

```
fig, ax1 = plt.subplots(figsize=(8,4))


# Left Y axis
ax1.plot(x, np.sin(x), 'b-', label='sin')
ax1.set_ylabel('sin(x)', color='blue')
ax1.tick_params(axis='y', labelcolor='blue’)

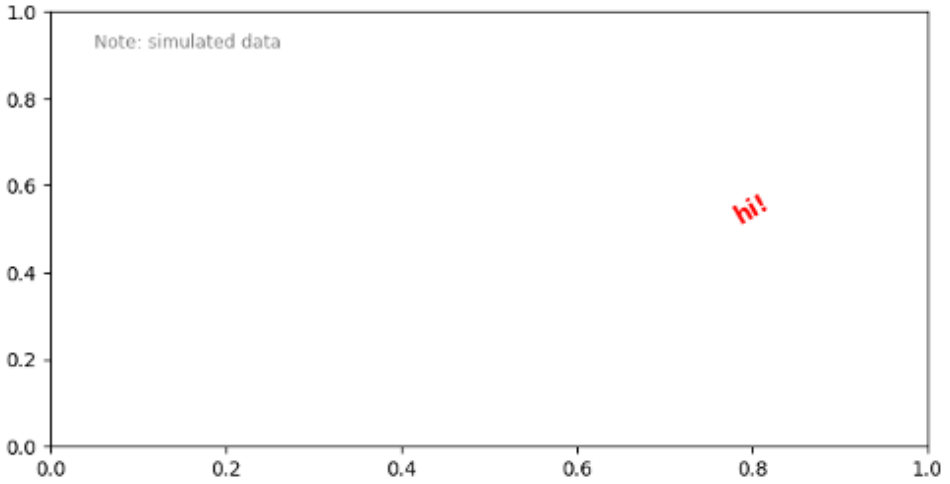

# Row Y axis (sharing X axis)
ax2 = ax1.twinx()
ax2.plot(x, np.exp(x/3), 'r--', label='exp')
```

Matplotlib – text

```
fig, ax = plt.subplots(figsize=(8,4))

# Text data coordinates
ax.text(0.8, 0.5, 'hi!',
        fontsize=12, color='red',
        ha='center', va='bottom',
        fontweight='bold',
        rotation=30)

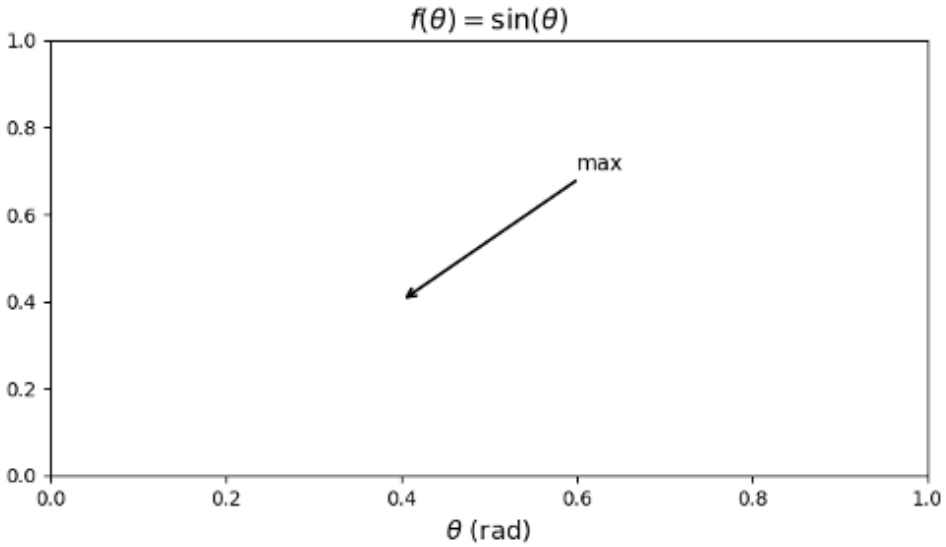
# Text in axis coordinates (0-1)
ax.text(0.05, 0.95,
        'Note: simulated data',
        transform=ax.transAxes,
        fontsize=9, color='grey',
        verticalalignment='top')
```



Matplotlib – text

```
fig, ax = plt.subplots(figsize=(8,4))

# Annotation with arrow
ax.annotate('max',
        xy=(0.4, 0.4), # arrow's head
        xytext=(0.6, 0.7), # text position
        fontsize=11,
        arrowprops=dict(arrowstyle='->',
                        lw=1.5))
# LaTeX text
ax.set_xlabel(r'$\theta$ (rad)',
              fontsize=13)
ax.set_title(r'$f(\theta)=\sin(\theta)$',
              fontsize=14)
```

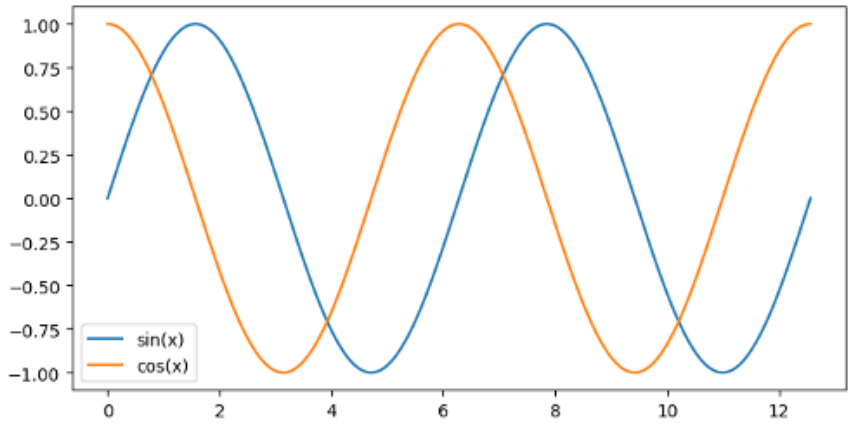


Matplotlib – legend

```
fig, ax = plt.subplots(figsize=(8,4))

x = np.linspace(0, 4*np.pi, 300)

# Auto legend with label=
ax.plot(x, np.sin(x), label='sin(x)')
ax.plot(x, np.cos(x), label='cos(x)')
ax.legend()
```

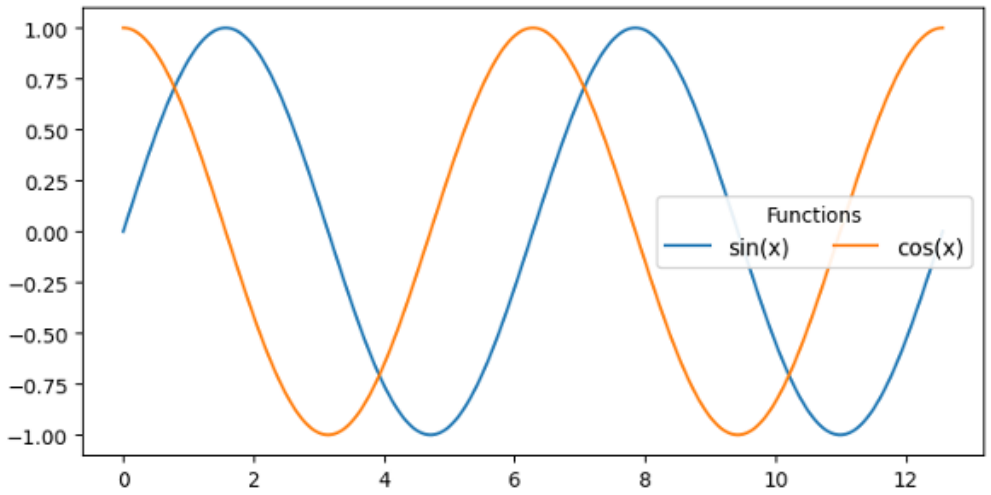


Matplotlib – legend

```
fig, ax = plt.subplots(figsize=(8,4))

x = np.linspace(0, 4*np.pi, 300)

# Auto legend with label=
ax.plot(x, np.sin(x), label='sin(x)')
ax.plot(x, np.cos(x), label='cos(x)')
ax.legend(loc='upper right', # best
          fontsize=11,
          title='Functions',
          framealpha=0.9,
          ncols=2)
```

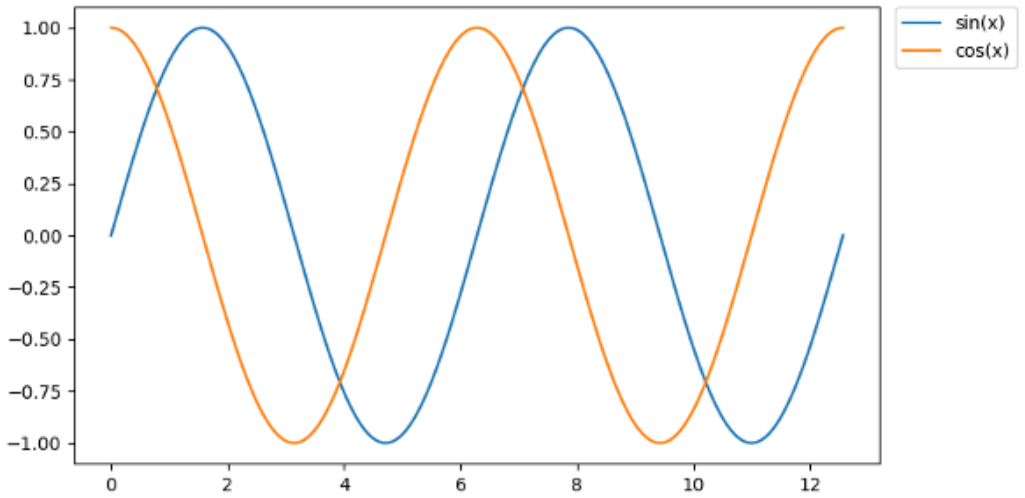


Matplotlib – legend

```
fig, ax = plt.subplots(figsize=(8,4))

x = np.linspace(0, 4*np.pi, 300)

# Auto legend with label=
ax.plot(x, np.sin(x), label='sin(x)')
ax.plot(x, np.cos(x), label='cos(x)')
ax.legend(bbox_to_anchor=(1.02, 1),
          loc='upper left',
          borderaxespad=0.)
fig.tight_layout() #
```

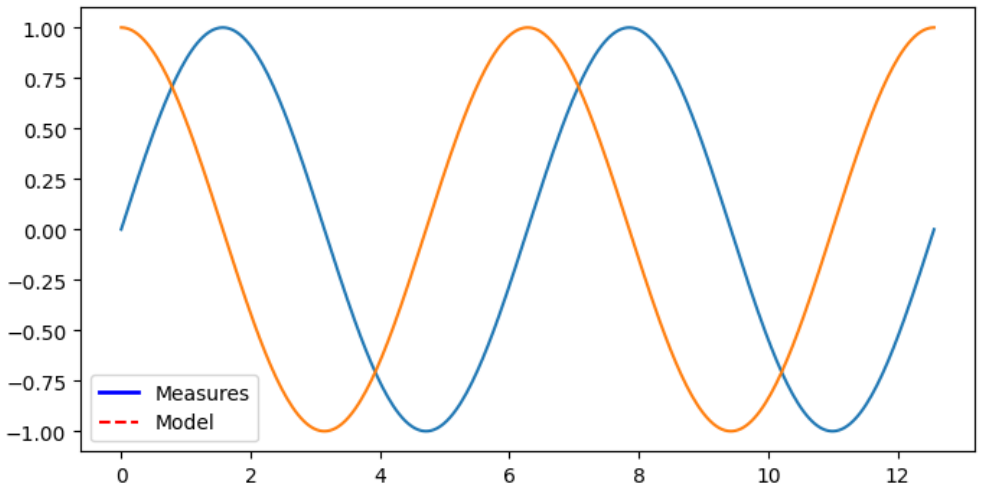


Matplotlib – legend

```
fig, ax = plt.subplots(figsize=(8,4))

x = np.linspace(0, 4*np.pi, 300)

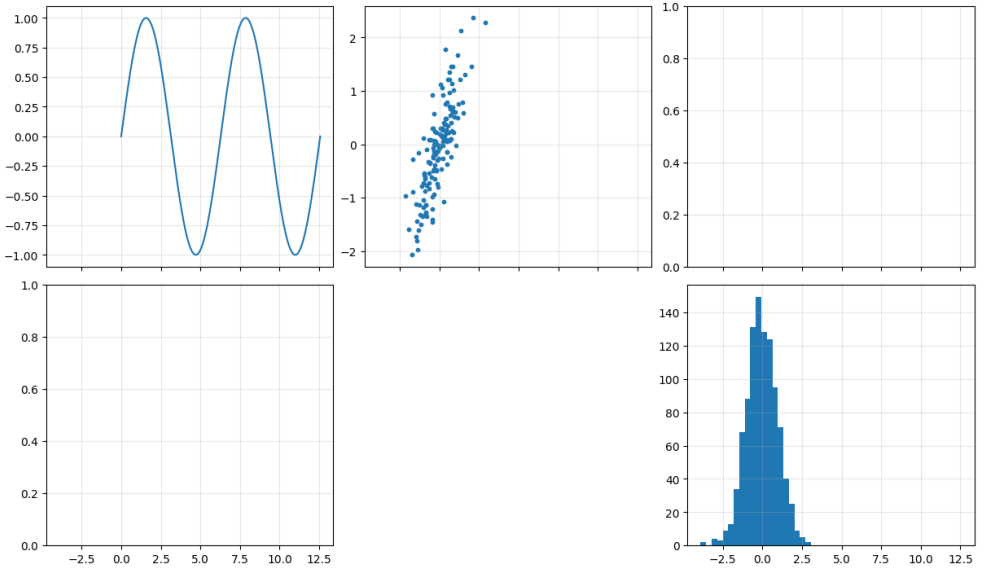
# Auto legend with label=
ax.plot(x, np.sin(x), label='sin(x)')
ax.plot(x, np.cos(x), label='cos(x)')
from matplotlib.lines import Line2D
handles = [
    Line2D([0],[0], color='b', lw=2,
        label='Measures'),
    Line2D([0],[0], color='r', ls='--',
        label='Model'),
    ]
ax.legend(handles=handles)
```



Matplotlib – subplots

```
# 2 rows × 3 cols
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(12, 7),
                         sharex=True, constrained_layout=True)
x = np.linspace(0, 4*np.pi, 300)
data = rng.standard_normal(1000)
# axes is an numpy array with shape (2, 3)
axes[0, 0].plot(x, np.sin(x))
axes[0, 1].scatter(x_s, y_s, s=10)
axes[1, 2].hist(data, bins=20)
# Iterate over all axes
for ax in axes.flat:
        ax.grid(True, alpha=0.3)


axes[1, 1].set_visible(False) # hide empty axis
```



Matplotlib

- Documentation: https://matplotlib.org/stable/
- Tutorials: https://matplotlib.org/stable/tutorials/
- Exemples and gallery: https://matplotlib.org/stable/gallery/
- Pyplot API reference: https://matplotlib.org/stable/api/pyplot_summary.html
- Axes API reference: https://matplotlib.org/stable/api/axes_api.html
- Cheat sheets: https://matplotlib.org/cheatsheets/
- mplot3d toolkit: https://matplotlib.org/stable/api/toolkits/mplot3d.html
- Colormap reference: https://matplotlib.org/stable/gallery/color/colormap_reference.html